

MATH70102 Unsupervised Learning — Coursework 1

Generative AI Usage

NotebookLM (Google) was used to query and cross-reference the course lecture notes and slides, helping identify the correct algorithmic approach for each problem (e.g. confirming the appropriate NMF update rule and the SMACOF convergence settings for MDS). **Claude** (Anthropic) was used to assist with typesetting this L^AT_EX document. All analysis, code, mathematical derivations, and written answers are the author's own work.

1 Denoising the Egyptian Goose with PCA and NMF

1.1 Problem and approach

We are given a clean and a noisy 512×512 greyscale image of an Egyptian goose, as pixel matrices X_{clean} and X_{noisy} . For a selection of integers q covering $1, \dots, 512$ we reconstruct the noisy image in two ways and measure how close each reconstruction is to the clean image in *Euclidean (Frobenius) distance*

$$d(A, B) = \sqrt{\sum_{i,j} (A_{ij} - B_{ij})^2}.$$

PCA (Week 2): treating the 512 rows as samples in $\mathbb{R}^{512}$, we keep the first q principal components and reconstruct. **NMF** (Week 3): we factorise $X_{\text{noisy}} \approx WH$ with $W \in \mathbb{R}_{\geq 0}^{512 \times q}$, $H \in \mathbb{R}_{\geq 0}^{q \times 512}$ and reconstruct via WH . Writing $d_0 = d(X_{\text{clean}}, X_{\text{noisy}})$ for the baseline, a method *denoises* if some q gives $d(X_{\text{clean}}, \hat{X}_q) < d_0$.

We load the data and define the distance. The noisy image is already strictly positive, so it is directly admissible to NMF without the +1.01 shift used in the Week 3 digit example (whose data lived in $[-1, 1]$ with zeros).

```
import matplotlib.pyplot as plt
import numpy as np
import numpy.typing as npt
from sklearn.decomposition import PCA

X_clean = np.loadtxt("egyptian_goose_clean.csv", delimiter=",")
X_noisy = np.loadtxt("egyptian_goose_noisy.csv", delimiter=",")

def euclid(A, B):
    # Euclidean / Frobenius distance
    return np.sqrt(np.sum((A - B) ** 2))

baseline = euclid(X_clean, X_noisy) # d_0 = 29.5459
```

1.2 Choosing the grid of q via a scree plot

We let the data guide the grid using a scree plot of the noisy image's principal components — the Week 2 tool for deciding how many components to retain. The variance is heavily front-loaded (PC1 alone $\approx 27\%$, PC2 $\approx 14\%$; 80% is reached at $q = 42$, 95% at $q = 188$), so we sample densely at low q , refine over the mid-band where most variance is captured, and keep a coarse tail to $q = 512$ (included as a check: there the reconstruction must equal the noisy image).

```
scree = PCA().fit(X_noisy)
evr = scree.explained_variance_ratio_
cum = np.cumsum(evr)

fig, axs = plt.subplots(1, 2, figsize=(8, 2.7))
axs[0].bar(range(1, 31), evr[:30] * 100, color="salmon")
axs[0].set_xlabel("Principal component"); axs[0].set_ylabel("Explained variance (%)")
axs[1].plot(range(1, len(cum) + 1), cum * 100, color="royalblue")
axs[1].set_xlabel("Number of components q"); axs[1].set_ylabel("Cumulative (%)")

q_grid = np.array(sorted(set(
    list(range(1, 21))
    + [25, 30, 35, 40, 45, 50, 55, 60, 65, 70, 75, 80, 90, 100]
    + [110, 125, 150, 175, 200, 225, 250, 275, 300]
    + [350, 400, 450, 500, 512])))
```

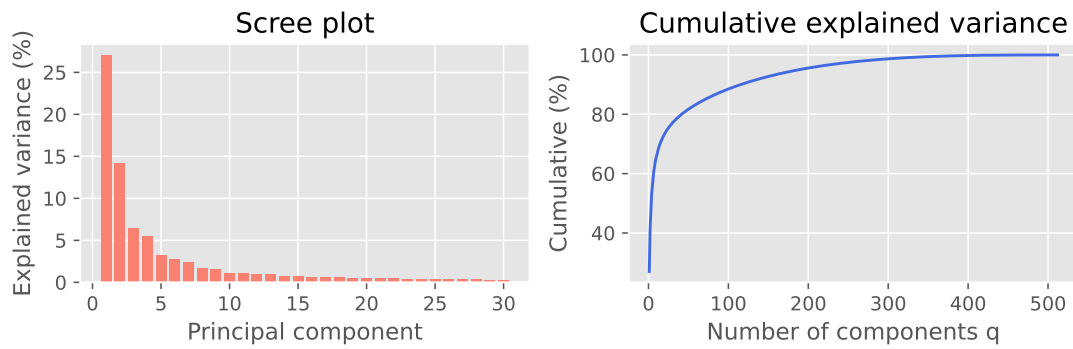


Figure 1: Scree plot (left) and cumulative explained variance (right) of the noisy image. The steep decay justifies dense sampling at low q .

1.3 PCA reconstruction (Week 2)

For each q we fit `PCA(n_components=q)`, project with `fit_transform`, and reconstruct with `inverse_transform`. PCA re-adds the sample mean, so at $q = 512$ the noisy image is recovered exactly.

```
pca_dist = np.empty(len(q_grid)); pca_recons = {}
for i, q in enumerate(q_grid):
    pca = PCA(n_components=int(q))
    recon = pca.inverse_transform(pca.fit_transform(X_noisy))
    pca_dist[i] = euclid(X_clean, recon)
    pca_recons[int(q)] = recon

best_pca_idx = int(np.argmin(pca_dist)); best_pca_q = int(q_grid[best_pca_idx])
# PCA: best q = 80 -> d = 24.9536; d at q=512 = 29.5459 (= baseline)
```

1.4 NMF reconstruction (Week 3)

We use the `NMF_numpy` class from the Week 3 reference *verbatim*. It maximises the quasi-likelihood $L(W, H) = \sum X \odot \log(WH) - WH$ by multiplicative updates; the reconstruction is stored in `.WH`.

```
class NMF_numpy:
    def train(self, X: npt.NDArray[np.number], q: int, n_iter: int = 100) -> None:
        d, n = X.shape
        self.W = np.random.uniform(low=0, high=1, size=(d, q)).astype(X.dtype)
        self.H = np.random.uniform(low=0, high=1 / q, size=(q, n)).astype(X.dtype)
        self.WH = self.W @ self.H
        self.logL = np.empty(n_iter, dtype=X.dtype)
        self.dist = np.empty(n_iter, dtype=X.dtype)
        for a in range(n_iter):
            self.W *= (X / self.WH) @ self.H.T / self.H.sum(axis=1)[None, :]
            self.WH = self.W @ self.H
            self.H *= (self.W.T @ (X / self.WH)) / self.W.sum(axis=0)[:, None]
            self.WH = self.W @ self.H
            self.logL[a] = np.sum(X * np.log(self.WH) - self.WH)
            self.dist[a] = np.sum((X - self.WH) ** 2)
```

A convergence check at $q = 20$ (Figure 2) shows the objective flattens well before 500 iterations, so we use `n_iter=400` in the sweep, with the random W, H initialisation seeded for reproducibility.

```
np.random.seed(0)
check = NMF_numpy(); check.train(X_noisy, q=20, n_iter=500)
fig, axs = plt.subplots(1, 2, figsize=(8, 2.4))
axs[0].plot(range(1, 501), check.logL, color="red"); axs[0].set_ylabel(r"$L(W, H)$")
axs[1].plot(range(1, 501), check.dist, color="blue"); axs[1].set_ylabel(r"$\|X - WH\|^2$")

np.random.seed(0)
nmf_dist = np.empty(len(q_grid)); nmf_recons = {}
for i, q in enumerate(q_grid):
    nmf = NMF_numpy(); nmf.train(X_noisy, int(q), n_iter=400)
    nmf_dist[i] = euclid(X_clean, nmf.WH)
    nmf_recons[int(q)] = nmf.WH

best_nmf_idx = int(np.argmin(nmf_dist)); best_nmf_q = int(q_grid[best_nmf_idx])
# NMF: best q = 275 -> d = 25.2179
```

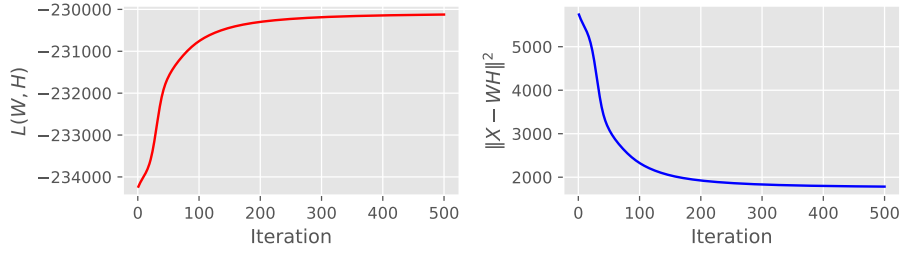


Figure 2: NMF convergence at $q = 20$: quasi-likelihood $L(W, H)$ (left) and reconstruction error $\|X - WH\|^2$ (right) against iteration. Both have plateaued by ~ 200 iterations.

1.5 Results

```
plt.plot(q_grid, pca_dist, "-o", label="PCA")
plt.plot(q_grid, nmf_dist, "-s", label="NMF")
plt.axhline(baseline, linestyle="--", label="Baseline = %.2f" % baseline)
plt.xlabel("q"); plt.ylabel("distance to clean image"); plt.legend()
```

	best q	$d(X_{\text{clean}}, \hat{X}_q)$	improvement vs. d_0	cum. variance at q	denoises?
Baseline (noisy)	—	29.55	—	—	—
PCA	80	24.95	4.59	86%	yes
NMF	275	25.22	4.33	98%	yes

Table 1: Best reconstructions. PCA beats the baseline for all $q \in [25, 500]$; NMF for all $q \in [35, 512]$.

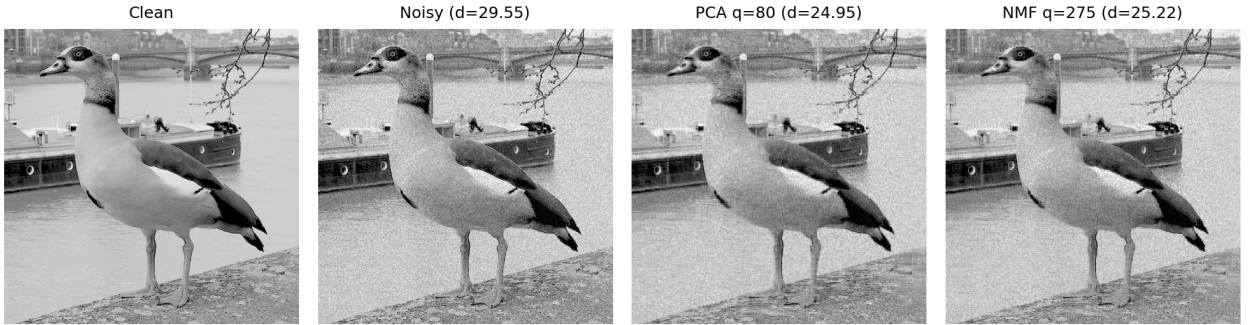
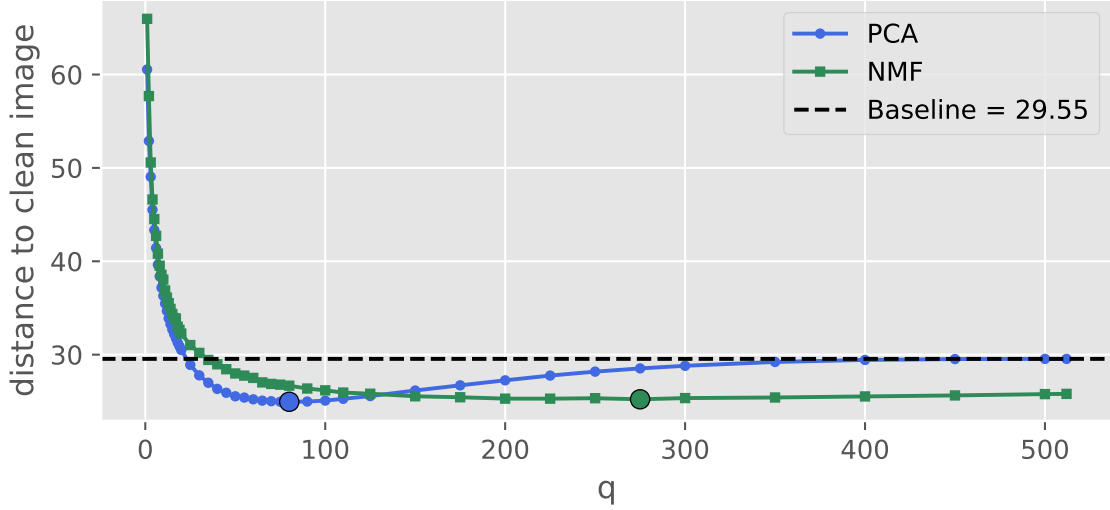


Figure 3: Top: distance to the clean image versus q ; both curves dip well below the baseline (dashed) and the marked points are the optima. Bottom: clean, noisy, and best- q PCA/NMF reconstructions (displayed on the clean image's intensity range).

1.6 Conclusion

Yes — both PCA and NMF denoise the image. For each method there is a wide band of q giving a reconstruction strictly closer to the clean image than the noisy image is: the distance falls from $d_0 = 29.55$ to 24.95 (PCA, $q \approx 80$) and 25.22 (NMF, $q \approx 275$). The distance-versus- q curve is **U-shaped**. Noise spreads its energy across all 512 directions, whereas the image's structure is concentrated in the leading components (scree plot: PC1 $\approx 27\%$). For small q the reconstruction under-fits and discards genuine structure; at an intermediate q it keeps the signal while rejecting the many noise-dominated high-index components — the denoising optimum; and as $q \rightarrow 512$ it recovers the noisy image ever more faithfully, so the distance climbs back to d_0 (for PCA this is exact, verified at $q = 512$). PCA reaches its optimum earlier and slightly lower than NMF, consistent with its components being the variance-optimal directions, so it concentrates the signal into fewer of them. The improvement is real but moderate ($\approx 15\%$): broadband noise can only be partly removed by a low-rank truncation, since only the portion of the noise lying in the discarded components is rejected.

2 Mapping Airports with Multidimensional Scaling

2.1 Problem and approach

We are given great-circle distances (km) between six northern-hemisphere airports (Table 2). The task is to *draw a map* — an embedding in $\mathbb{R}^2$ that reproduces these distances as faithfully as possible in relative terms. This is exactly the setting for **multidimensional scaling (MDS)** (Week 4): given only pairwise dissimilarities d_{ij} , MDS places points $z_1, \dots, z_n$ in $\mathbb{R}^q$ to minimise the *least-squares stress*

$$S_M(z_1, \dots, z_n) = \sum_{i \neq j} (d_{ij} - \|z_i - z_j\|)^2.$$

We use `sklearn.manifold.MDS` with `dissimilarity="precomputed"`, following the Week 4 reference solution.

	CPH	EWR	KEF	LHR	NRT
EWR	6 223				
KEF	2 151	4 185			
LHR	981	5 577	1 900		
NRT	8 734	10 833	8 844	9 614	
YVR	7 685	3 908	5 702	7 601	7 521

Table 2: Great-circle distances (km). CPH = Copenhagen, EWR = Newark, KEF = Keflavík, LHR = London Heathrow, NRT = Tokyo Narita, YVR = Vancouver.

We build the symmetric 6×6 dissimilarity matrix D directly from the lower triangle of the table.

```
import warnings
import numpy as np, matplotlib.pyplot as plt
from sklearn.manifold import MDS
from sklearn.metrics import pairwise_distances

warnings.filterwarnings("ignore", category=FutureWarning)
plt.style.use("ggplot")

labels = ["CPH", "EWR", "KEF", "LHR", "NRT", "YVR"]
lower = [[6223], [2151, 4185], [981, 5577, 1900], [8734, 10833, 8844, 9614], [7685, 3908, 5702, 7601, 7521]]
n = len(labels); D = np.zeros((n, n))
for i, row in enumerate(lower, start=1):
    for j, d in enumerate(row): D[i, j] = D[j, i] = d
```

2.2 Part (a) — the 2-D map

We minimise S_M in $\mathbb{R}^2$ using MDS. The orientation of the result is arbitrary: stress is invariant under rotation, reflection and translation. Because SMACOF (the underlying algorithm) minimises a non-convex objective, we use 300 random restarts and tight convergence tolerances (`n_init=300`, `max_iter=1000`, `eps=1e-10`) to avoid poor local minima, keeping the configuration with the lowest stress.

```
mds2 = MDS(n_components=2, dissimilarity="precomputed", random_state=1234,
           normalized_stress=False, init="random", n_init=300,
           max_iter=1000, eps=1e-10)
Z = mds2.fit_transform(D)

plt.figure(figsize=(5, 4))
plt.scatter(Z[:, 0], Z[:, 1], color="black")
for (x, y), name in zip(Z, labels):
```

```
plt.text(x, y + 120, name, fontsize=11, ha="center")
plt.gca().set_aspect("equal"); plt.margins(0.18)
plt.xlabel("axis 1 (km)"); plt.ylabel("axis 2 (km)")
plt.title(r"MDS map of airports ( $\mathbb{R}^2$ )"); plt.tight_layout(); plt.show()
```

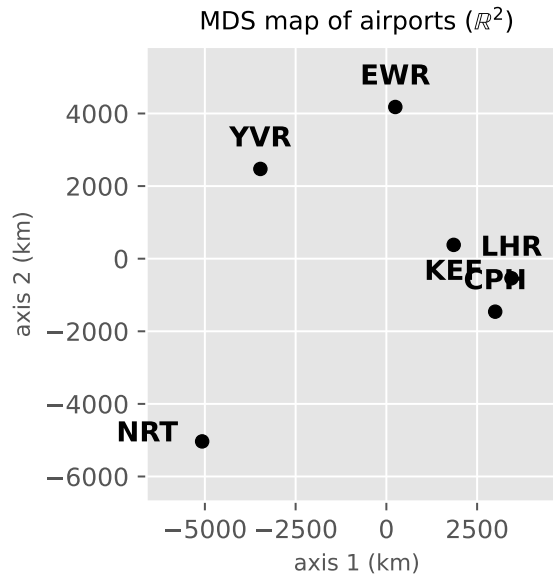


Figure 4: MDS embedding of the six airports in $\mathbb{R}^2$. The relative layout is consistent with real geography up to an arbitrary rotation/reflection: the European cluster CPH–KEF–LHR sits together (with KEF reaching towards the Atlantic), EWR lies across the Atlantic, YVR is on the Pacific side of North America, and NRT (Tokyo) is the most isolated point.

2.3 Part (b) — accuracy of the map

We report the raw stress S_M and the scale-free *Kruskal stress-1*, $\text{stress-1} = \sqrt{S_M / \sum_{i < j} d_{ij}^2}$, and draw a Shepard diagram comparing embedded distances to true distances.

```
iu = np.triu_indices(n, 1)
d_true = D[iu]; d_map = pairwise_distances(Z)[iu]
kruskal = np.sqrt(mds2.stress_ / np.sum(d_true ** 2))
print("S_M = %.4e km^2, Kruskal stress-1 = %.4f (%.2f%%)"
      % (mds2.stress_, kruskal, 100 * kruskal))
# S_M = 1.5086e+05 km^2, Kruskal stress-1 = 0.0149 (1.49%)

lim = d_true.max() * 1.07
plt.figure(figsize=(3.5, 3.5))
plt.plot([0, lim], [0, lim], "k-", lw=1)
plt.scatter(d_true, d_map, color="salmon", edgecolor="k")
plt.xlabel("true distance (km)"); plt.ylabel("map distance (km)")
plt.title("Shepard diagram"); plt.tight_layout(); plt.show()
```

The Kruskal stress-1 is 1.49% and the largest pairwise error is 194 km (against distances up to 10 833 km), confirming the map is accurate in relative terms. It **cannot be perfectly accurate** because the given values are *great-circle (geodesic) distances on the sphere* $\mathbb{S}^2$. As the Week 4 slides illustrate ($\mathbb{S}^2 \rightarrow \mathbb{R}^2$), a curved 2-manifold cannot be flattened into the Euclidean plane without distorting distances, so MDS's flat proxy always leaves a non-zero stress.

2.4 Part (c) — embedding in $\mathbb{R}^d$ for $d > 2$

We repeat MDS for $d = 1, \dots, 5$ (with $n = 6$ points, at most $n - 1 = 5$ dimensions are useful) with identical settings and plot the minimised stress.

```
dims = list(range(1, 6)); stresses = []
for d in dims:
    m = MDS(n_components=d, dissimilarity="precomputed", random_state=1234,
            normalized_stress=False, init="random", n_init=300,
            max_iter=1000, eps=1e-10)
    m.fit_transform(D); stresses.append(m.stress_)
plt.figure(figsize=(4, 2.8))
plt.plot(dims, stresses, "-o", color="royalblue"); plt.xticks(dims)
plt.xlabel("dimension d"); plt.ylabel("stress S_M (km^2)")
plt.title("Stress vs. embedding dimension"); plt.tight_layout(); plt.show()
# d=1: 4.54e7, d=2: 1.51e5, d=3: 1.45e5, d=4,5: 1.45e5 (plateau)
```

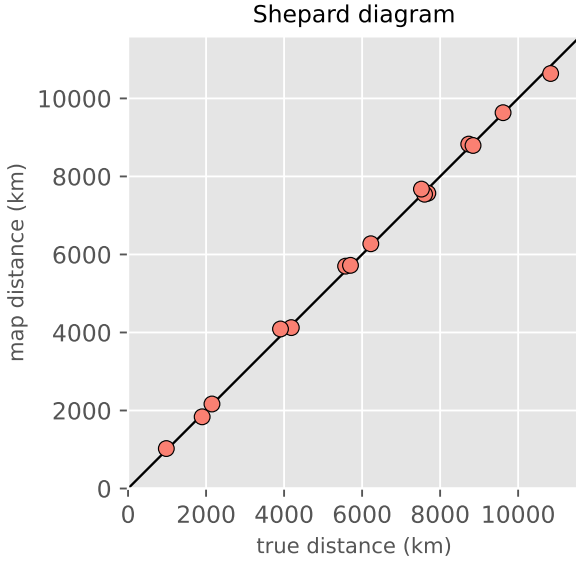


Figure 5: Shepard diagram for the $\mathbb{R}^2$ map. Points lie close to the diagonal, confirming a good relative fit.

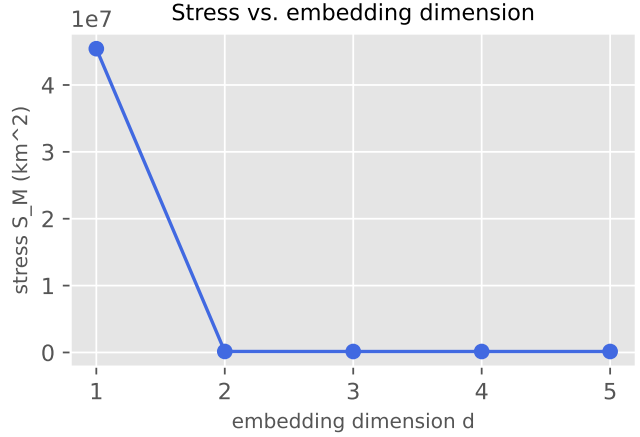


Figure 6: Minimum stress S_M vs. embedding dimension d . Stress falls sharply from $d = 1$ to $d = 2$, improves slightly at $d = 3$, then plateaus and stays strictly positive.

2.5 Conclusion

Increasing d adds degrees of freedom, so the minimum stress is **non-increasing** in d and accuracy improves — from $d = 1$ (stress 4.54×10^7) to $d = 2$ (1.51×10^5 , Kruskal 1.49%) to $d = 3$ (1.45×10^5), after which it **plateaus**: $d = 4$ and $d = 5$ give essentially the same stress as $d = 3$. The plateau reflects the intrinsic geometry: the airports lie on the 2-sphere, so three Euclidean dimensions already capture essentially all the embeddable structure.

Crucially, the stress **never reaches zero**, even at $d = 5$. Great-circle distances on a sphere are *not* Euclidean distances in any dimension; no finite-dimensional Euclidean embedding can be exact and a non-zero irreducible stress always remains. The practical gain beyond $\mathbb{R}^2$ is small, so the 2-D map of Part (a) is already a faithful relative representation.

3 PCA cannot, but ICA can, recover mixed sources

3.1 Problem and construction rationale

We build three one-dimensional signals x_1, x_2, x_3 (sine, square, sawtooth), form mixtures $Y = AX$ via a fixed 3×3 matrix A , and ask whether PCA or ICA can recover X from Y alone.

Two design choices make the contrast sharp. **(1) Non-Gaussian, independent sources.** ICA assumes at most one Gaussian source; rotating independent Gaussians produces independent Gaussians again, so a purely Gaussian model is unidentifiable. Our three waveforms—sine (arcsine distribution), square (bimodal), sawtooth (uniform)—are manifestly non-Gaussian and independent. **(2) Non-orthogonal A .** PCA finds orthogonal variance directions. If A is not orthogonal, the true source directions in observation space are not orthogonal and PCA’s axes cannot align with them—the principal components remain mixtures. ICA fixes the rotation PCA leaves undetermined by maximising statistical independence (higher-order statistics), recovering the sources up to the known caveats of order, sign, and scale.

```
import numpy as np, matplotlib.pyplot as plt
from scipy import signal
from scipy.optimize import linear_sum_assignment
from sklearn.decomposition import PCA, FastICA

rng = np.random.default_rng(0)
n = 2000; t = np.linspace(0, 8, n)
x1 = np.sin(2 * t)           # sine
x2 = np.sign(np.sin(3 * t))  # square
x3 = signal.sawtooth(2 * np.pi * t) # sawtooth
X = np.vstack([x1, x2, x3])
X = X + 0.02 * rng.standard_normal(X.shape)
X = (X - X.mean(axis=1, keepdims=True)) / X.std(axis=1, keepdims=True)

A = np.array([[1.0, 1.0, 1.0],
               [0.5, 2.0, 1.0],
```

```

Y = A @ X
[1.5, 1.0, 2.0]]) # non-orthogonal: A @ A.T != c*I

```

3.2 PCA on the mixtures (fails)

```

pca = PCA(n_components=3)
Z = pca.fit_transform(Y.T)
Z = (Z / np.max(np.abs(Z), axis=0)).T # normalise scale; back to 3 x n

def corr_matrix(R, S):
    M = np.zeros((R.shape[0], S.shape[0]))
    for i in range(R.shape[0]):
        for j in range(S.shape[0]):
            M[i, j] = np.abs(np.corrcoef(R[i], S[j])[0, 1])
    return M

C_pca = corr_matrix(Z, X)
# |corr| matrix (rows = PC1-3, cols = x1,x2,x3):
# [[0.516 0.612 0.601]
#  [0.407 0.782 0.437]
#  [0.753 0.119 0.670]]

```

No principal component is strongly aligned with a single source: every row mixes substantial correlation across several sources and no clean one-to-one matching exists. The best $|\text{corr}|$ each PC achieves with any source is 0.61, 0.78, 0.75—all well below 1. This is expected: PCA can only decorrelate and must return *orthogonal* components, but the columns of A are not orthogonal, so its axes cannot recover the source directions.

3.3 ICA on the mixtures (succeeds)

```

ica = FastICA(n_components=3, whiten="unit-variance", random_state=0, max_iter=1000)
S = ica.fit_transform(Y.T)
S = (S / np.max(np.abs(S), axis=0)).T

C_ica = corr_matrix(S, X)
# |corr| matrix (rows = IC1-3, cols = x1,x2,x3):
# [[0.997 0.030 0.069]
#  [0.055 0.015 0.997]
#  [0.058 0.999 0.037]]
row, col = linear_sum_assignment(-C_ica)
order = col[np.argsort(row)]
matched = C_ica[np.arange(3), order] # [0.9968, 0.9969, 0.9994]

```

Each IC has $|\text{corr}| \approx 1$ with exactly one source and ≈ 0 with the other two. The optimal assignment maps $\text{IC1} \rightarrow x_1$, $\text{IC2} \rightarrow x_3$, $\text{IC3} \rightarrow x_2$, with matched $|\text{corr}| = 0.997, 0.997, 0.999$ and maximum recovery error $1 - |\text{corr}| = 0.003$.

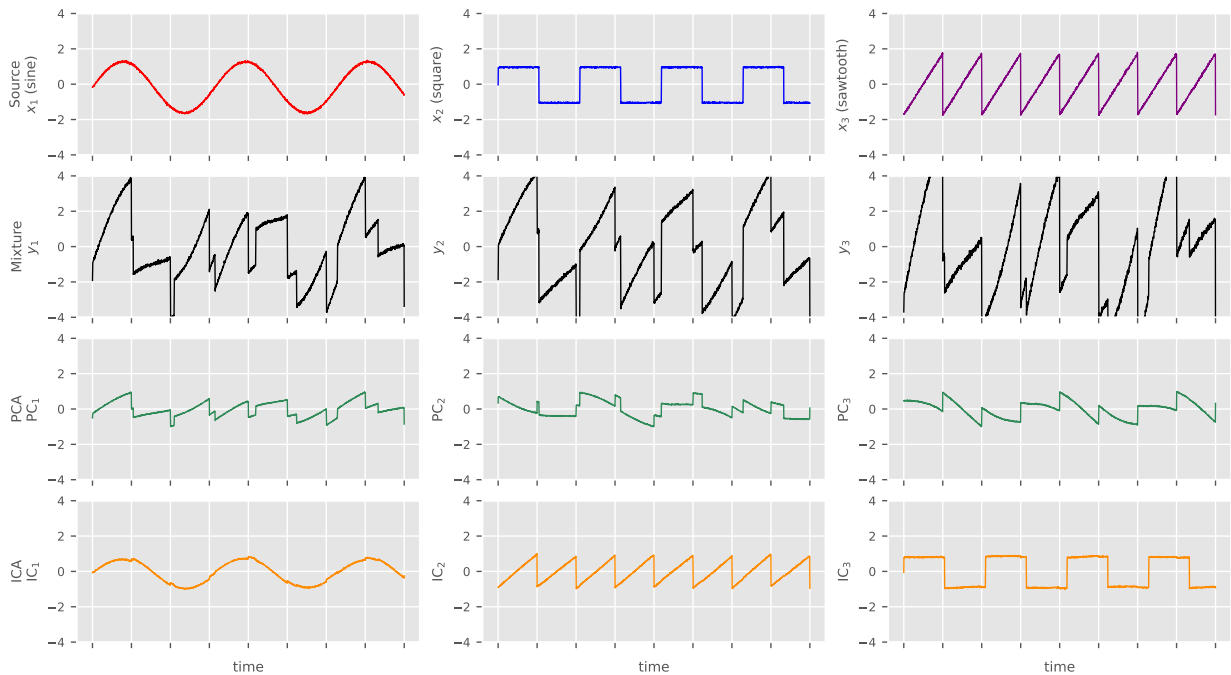


Figure 7: Row 1: original sources; Row 2: observed mixtures $Y = AX$; Row 3: principal components (still mixed); Row 4: independent components (source shapes restored).

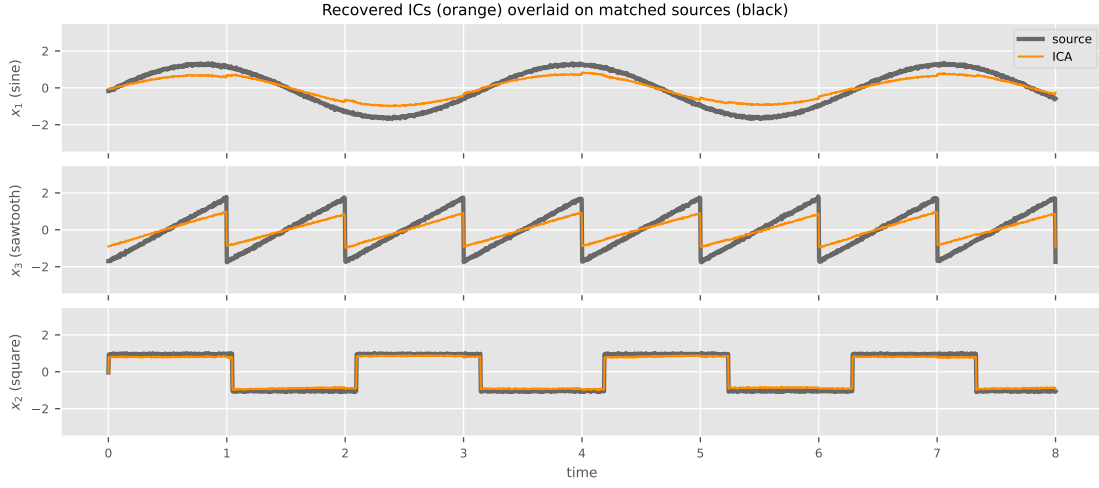


Figure 8: ICA recovery: each independent component (orange) overlaid on its matched source (black). The waveforms are virtually indistinguishable, confirming $|\text{corr}| \approx 1$.

3.4 Conclusion

Method	Criterion	Best $ \text{corr} $	Recovers sources?
PCA	orthogonal max-variance directions (2nd-order)	0.61, 0.78, 0.75	No
ICA	statistical independence / non-Gaussianity	1.00, 1.00, 1.00	Yes

Table 3: PCA vs. ICA on the cocktail-party problem.

PCA leaves a rotation of the whitened space undetermined—that rotation is invisible to second-order statistics. Since A is not orthogonal, the source axes are not orthogonal, so PCA’s axes cannot coincide with them and the principal components remain mixed. ICA resolves the undetermined rotation by demanding maximally independent/non-Gaussian components; because at most one source is Gaussian, this uniquely identifies the sources up to the unavoidable permutation, sign, and scale ambiguity—confirmed empirically by $|\text{corr}| \approx 1$.

4 Distance Concentration and the Curse of Dimensionality

4.1 Problem and simulation

We study how the ratio

$$R := \frac{\max_{i \neq j} \|X_i - X_j\|}{\min_{i \neq j} \|X_i - X_j\|}, \quad X_1, \dots, X_{10} \stackrel{\text{i.i.d.}}{\sim} N_d(0, I_d),$$

behaves as $d \rightarrow \infty$, averaged over 200 independent realisations per d .

```
import numpy as np, matplotlib.pyplot as plt
from scipy.spatial.distance import pdist

rng = np.random.default_rng(42)
d_values = [2, 5, 10, 50, 100, 500, 1_000, 5_000, 10_000, 50_000, 100_000]
n_rep, n_pts = 200, 10

R_means, R_stds = [], []
for d in d_values:
    ratios = [pdist(rng.standard_normal((n_pts, d))).max()
              / pdist(rng.standard_normal((n_pts, d))).min()
              for _ in range(n_rep)]
    R_means.append(np.mean(ratios)); R_stds.append(np.std(ratios))
```

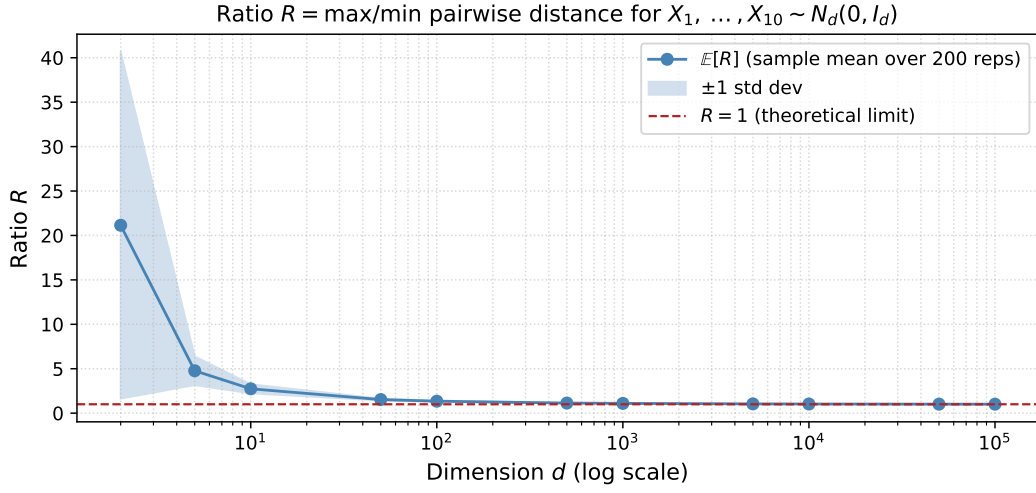



Figure 9: Mean ± 1 std of R over 200 repetitions as a function of d (log scale). $R \rightarrow 1$ rapidly; standard deviation also collapses, showing tight concentration.

4.2 Theoretical explanation (LLN and CLT)

Fix any pair (i, j) . Since $X_{i,k}, X_{j,k} \stackrel{\text{i.i.d.}}{\sim} N(0, 1)$, the differences $Z_k := X_{i,k} - X_{j,k} \sim N(0, 2)$, giving

$$\|X_i - X_j\|^2 = \sum_{k=1}^d Z_k^2, \quad \mathbb{E}[Z_k^2] = 2, \quad \text{Var}(Z_k^2) = 8.$$

By the **Law of Large Numbers**

$$\frac{\|X_i - X_j\|^2}{2d} \xrightarrow{p} 1 \implies \frac{\|X_i - X_j\|}{\sqrt{2d}} \xrightarrow{p} 1 \quad \text{as } d \rightarrow \infty.$$

The **CLT** gives the rate: $(\|X_i - X_j\|^2 - 2d)/\sqrt{8d} \rightarrow N(0, 1)$, so fluctuations around $\sqrt{2d}$ are of order $d^{-1/2} \rightarrow 0$. There are only $\binom{10}{2} = 45$ pairs—a fixed finite number independent of d . By a union bound, *all* 45 distances concentrate around the same value $\sqrt{2d}$ simultaneously, so the maximum and minimum converge to the same limit and

$$R = \frac{\max_{i \neq j} \|X_i - X_j\|}{\min_{i \neq j} \|X_i - X_j\|} \xrightarrow{p} 1 \quad \text{as } d \rightarrow \infty.$$

The histograms below (200 repetitions, all 45 distances per repetition, normalised by $\sqrt{2d}$) confirm the increasing concentration around 1.

```
d_check = [10, 100, 1_000, 10_000]; n_rep2 = 500
for d in d_check:
    all_norm = [v for _ in range(n_rep2)
                for v in (pdist(rng.standard_normal((n_pts, d))) / np.sqrt(2 * d)).tolist()]
    # histogram of all_norm concentrates at 1 as d grows
```

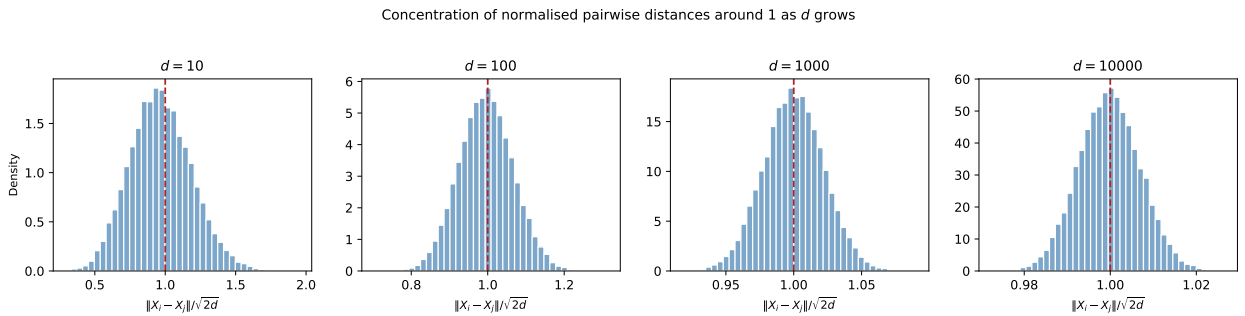


Figure 10: Histograms of $\|X_i - X_j\|/\sqrt{2d}$ over 500 repetitions $\times$ 45 pairs at four values of d . The distribution tightens around 1, with variance $O(1/d)$ as predicted.

4.3 Relation to the curse of dimensionality

When $R \approx 1$, every pair of points is roughly equidistant—the concept of a *nearest neighbour* becomes meaningless because “near” and “far” are indistinguishable. This breaks many unsupervised learning algorithms.

- ***K*-means / *K*-medoids (Week 3).** Cluster assignment picks the closest centroid. If all distances are nearly equal, every point is equidistant from all centroids and assignments become essentially arbitrary.
- **MDS (Week 4).** MDS preserves pairwise distances. When all distances concentrate at $\sqrt{2d}$, the data look like points on a sphere and a meaningful low-dimensional embedding ceases to exist.

This is the key motivation for dimensionality reduction *before* applying distance-based methods: techniques such as PCA (Weeks 2–3) and ICA (Week 2) project the data onto a lower-dimensional subspace where distances remain informative and R is bounded away from 1.